

Local Market Lab — Technical Documentation v0.8.0

*Privacy-first portfolio analytics on your own machine. No cloud. No data sharing.
No signals. No advice.*

Table of Contents

1. [Introduction & Mission](#)
2. [Architecture Overview](#)
3. [Quick Start](#)
4. [API Reference](#)
5. [Portfolio Engine](#)
6. [Backtest Engine](#)
7. [Scenario Engine](#)
8. [Trading Game](#)
9. [AI Prediction](#)

10. [Risk Analytics](#)
 11. [Bank-Ready & Compliance](#)
 12. [Windows App](#)
 13. [Ollama Integration](#)
 14. [Configuration](#)
 15. [FAQ](#)
 16. [Troubleshooting](#)
-

1. Introduction & Mission

Local Market Lab is a locally-running, privacy-focused workbench for:

- **Portfolio analytics** — Valuation, P&L, allocation
- **Backtesting** — Strategy testing with fees, slippage, benchmarks
- **Scenario simulation** — Monte Carlo, Block-Bootstrap, historical replay
- **Trading game** — Paper trading with virtual capital and leaderboard

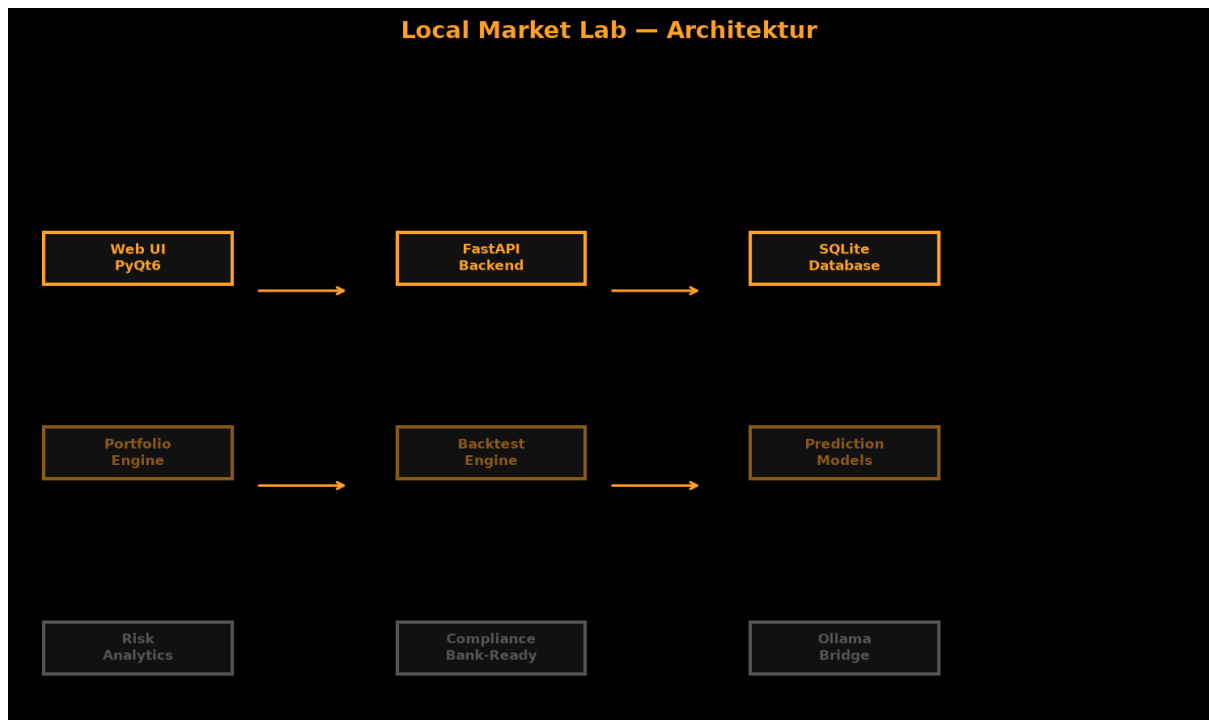
- **AI prediction** — 15+ models, locally, without cloud dependency
- **Risk analytics** — VaR, CVaR, correlation, rolling metrics

Mission: Make institutional methodology available to private users without compromising on privacy and reproducibility.

Design Principles:

Principle	Implementation
Local	Everything runs locally. No cloud required.
Privacy	No telemetry, no external service dependencies
Reproducibility	Every calculation has seed, timestamp, data hash
Transparency	All methods documented, no black-box models
Decimal-Only	No floats for monetary amounts, ROUND_HALF_UP
Determinism	Same inputs → same outputs

2. Architecture Overview



2.1 Layers

Layer	Technology	Purpose
Web UI	FastAPI + HTML/Canvas	Bloomberg-terminal-style browser interface
Windows App	PyQt6 + pyqtgraph	Native desktop app with real-time charts
API Backend	FastAPI	REST + WebSocket, port 8322
Database	SQLite	Append-only where relevant, isolated per user
Python Packages	numpy, requests, sqlite3	Calculations without external dependencies

2.2 Module Structure

```
local-market-lab/
├── apps/
│   ├── api/          # FastAPI REST and WebSocket server
│   ├── cli/          # Typer command-line tool
│   ├── web/          # HTML/CSS/Canvas terminal UI
│   ├── mcp/          # Model Context Protocol server
│   └── gui/          # PyQt6 desktop app (Windows)
├── packages/
│   ├── core/         # Money, Dates, Hashing
│   ├── domain/       # Entities (Instrument, Transaction, ...)
│   ├── storage/      # SQLite workspace, state singletons
│   ├── ingest/       # CSV import, demo data
│   ├── marketdata/  # Price series, FX, adapters
│   ├── portfolio/    # Position engine, valuation
│   ├── metrics/      # CAGR, Sharpe, Sortino, VaR, CVaR
│   ├── backtest/     # Event loop, strategies
│   ├── scenarios/    # Monte Carlo, bootstrap, prediction
│   ├── artifacts/    # Reproducibility manifests
│   ├── compliance/   # Guard, audit, bank-ready
│   ├── reports/      # Markdown reports
│   ├── game/         # Trading game, multiplayer lobby
│   └── ollama/       # Local LLM bridge
├── tests/            # 90+ pytest tests
└── docs/             # Documentation and graphics
```

3. Quick Start

3.1 Installation

```
# Clone repository
git clone https://github.com/mrmixx-max/local-market-lab.git
cd local-market-lab

# Python 3.10+ required
python --version

# Install dependencies
pip install -e ".[dev]"
```

3.2 First Start (Web UI)

```
# Start API server
python -m apps.api
|
# Open browser
start http://127.0.0.1:8322/
```

3.3 First Start (Windows App)

```
# Run build script
cd windows/src && python build.spec
|
# Or start directly
python -m windows.src.app
```

3.4 Windows Installer

Download `LocalMarketLab-Setup-v0.8.0.exe` from releases, run, follow instructions.

4. API Reference

4.1 Health Check

```
GET /api/v1/health
```

Response:

```
{
  "status": "ok",
  "instruments": 4,
  "version": "0.8.0",
  "db_connected": true,
  "uptime_seconds": 42.5
}
```

4.2 Market Data

```
GET /api/v1/market/symbols
GET /api/v1/market/prices/{symbol}
```

4.3 Portfolio

```
GET /api/v1/portfolio/{name}?benchmark=IWDA&include_analytics=true
```

4.4 Backtest

```
POST /api/v1/backtest
```

```
{
  "portfolio": {
    "IWDA": 0.6,
    "EIMI": 0.4
  },
  "strategy": "buy_and_hold",
  "fees_bps": 5,
  "slippage_bps": 2
}
```

4.5 Scenario

```
POST /api/v1/scenario
```

```
{  
  "symbols": ["IWDA", "EIMI", "AGGH"],  
  "method": "monte_carlo",  
  "n_simulations": 10000,  
  "horizon_days": 252,  
  "initial_value": 100000,  
  "seed": 42  
}
```

4.6 AI Prediction

```
POST /api/v1/scenario/forecast/{symbol}
```

```
{  
  "horizon": 30,  
  "method": "ensemble"  
}
```

4.7 Trading Game

```
POST /api/v1/game/create  
POST /api/v1/game/{game_id}/order  
POST /api/v1/game/{game_id}/tick  
GET /api/v1/game/{game_id}/state  
GET /api/v1/game/leaderboard
```

4.8 Compliance

```
GET /api/v1/compliance/audit-log  
POST /api/v1/compliance/integrity-check  
GET /api/v1/compliance/report
```

5. Portfolio Engine

5.1 Valuation

The portfolio engine uses exclusively `Decimal` values for monetary amounts. Float inputs are rejected during parsing.

FX Policy: Missing exchange rates produce an `incomplete` state — never silent 1:1 conversion.

5.2 Corporate Actions

- **Splits:** Automatic quantity adjustment
- **Dividends:** Cash dividends as separate transactions
- **Chronological order:** All actions processed in order

5.3 Allocation

```
GET /api/v1/portfolio/{name}?include_analytics=true
```

Returns `allocation` by `asset_class` from the instruments table.

6. Backtest Engine

6.1 Available Strategies

Strategy	Description
buy_and_hold	Buy once, never sell
periodic_rebalance_63	Rebalance every 63 days (quarterly)
momentum_20	20-day momentum, buy top asset
mean_reversion_20	20-day mean reversion

6.2 Metrics

- **CAGR**: Compounded Annual Growth Rate (252-day annualized)
- **Max Drawdown**: Maximum loss from peak
- **Sharpe Ratio**: Risk-adjusted return ($R_f=0$)
- **Sortino Ratio**: Like Sharpe, but only downside volatility
- **Calmar Ratio**: CAGR / Max Drawdown

7. Scenario Engine

7.1 Methods

Method	Seed	Description
Monte Carlo	yes	iid normal returns
Block-Bootstrap	yes	Random blocks from historical returns
Historical-Replay	yes	Use one historical year

7.2 Output

All methods return: - Percentiles (P05, P25, P50, P75, P95) - Loss probability - Disclaimer:
No forecast, only scenario

8. Trading Game

8.1 Challenges

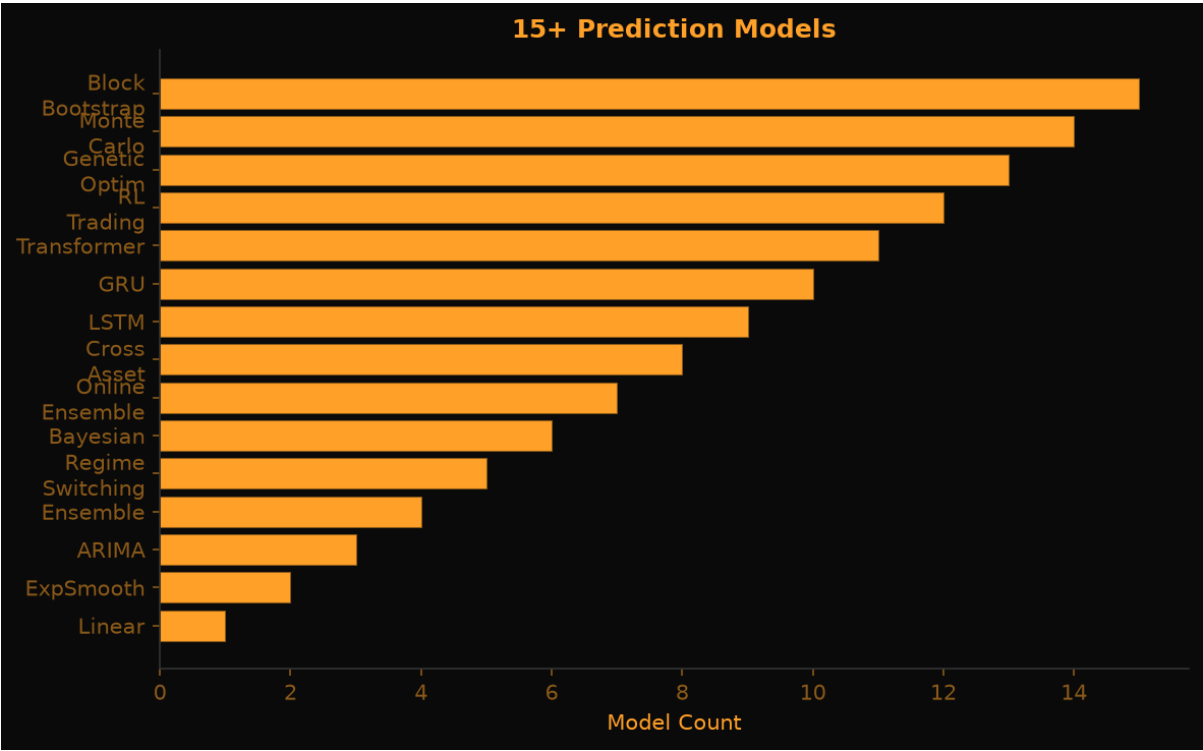
Challenge	Goal
<code>beat_market</code>	Beat benchmark (IWDA)
<code>low_volatility</code>	Max volatility < 10%
<code>income_generator</code>	Monthly dividends
<code>max_sharpe</code>	Max Sharpe ratio
<code>min_volatility</code>	Min portfolio volatility
<code>beat_benchmark_by_5pct</code>	5% above benchmark

8.2 Leaderboard

Endgame summary per game: - Total return, CAGR, Max Drawdown, Sharpe, Sortino -
Number of trades, Win rate - Equity curve for comparison

9. AI Prediction

9.1 15+ Models



Category	Models
Basic	Linear Trend, Holt's ExpSmooth, ARIMA-like, Ensemble

Advanced	Regime-Switching, Bayesian Trend/Seasonal, Online Ensemble, Cross-Asset
Deep Learning	LSTM, GRU (BPTT, Adam, Gradient Clipping)
Reinforcement Learning	Q-Learning, DQN, REINFORCE
Genetic Optimization	Feature Selection, Differential Evolution, NSGA-II
Scenarios	Monte Carlo, Block-Bootstrap, Historical-Replay

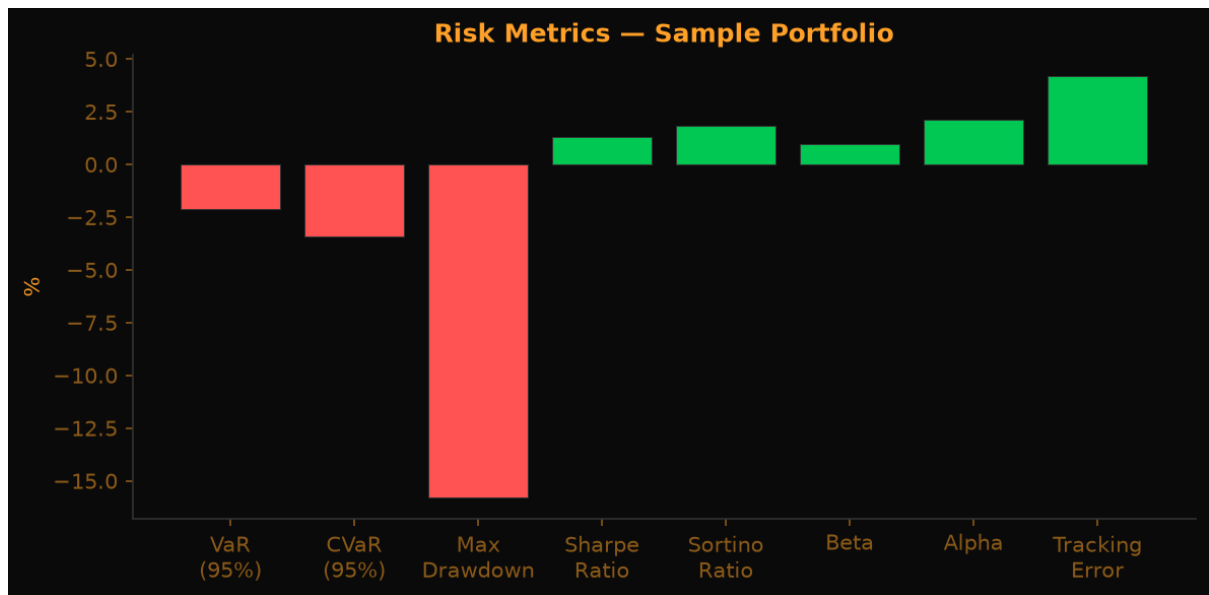
9.2 Ensemble

`ensemble_forecast()` combines three basic models weighted by inverse variance.

9.3 Confidence Intervals

All models return 68% and 95% credible/confidence intervals.

10. Risk Analytics



10.1 Metrics

- **VaR (95%)**: Value at Risk, historical simulation
- **CVaR (95%)**: Expected Shortfall
- **Rolling Sharpe**: 63-day rolling
- **Drawdown Series**: Drawdown as time series
- **Performance Attribution**: Per-position return contribution
- **Correlation Matrix**: Pearson correlation between positions

11. Bank-Ready & Compliance

11.1 Audit Logger

Append-only log of all API actions: - User, Timestamp, Action, Params, Result-Hash - SHA-256 checksums per table snapshot

11.2 Data Integrity

Automatic checksums for `instruments`, `transactions`, `prices`, `corporate_actions`.

11.3 Compliance Report (BaFin-Style)

```
{
  "system_version": "0.8.0",
  "audit_log_summary": {...},
  "data_integrity_status": "valid",
  "user_actions_count": 42,
  "risk_flags": []
}
```

11.4 GDPR Export & Deletion

- `GET /api/v1/compliance/export/{user}` — JSON export
- `POST /api/v1/compliance/delete-account` — Anonymization

12. Windows App

12.1 Installation

1. Download LocalMarketLab-Setup-v0.8.0.exe
2. Run installer
 - Desktop shortcut optional
3. Start app

12.2 Features

- **6 Tabs**: Markets, Backtest, Scenarios, Game, Ollama, Risk
- **Sidebar**: Watchlist with live updates
- **Top Bar**: Branding, clock, connection status
- **Status Bar**: Disclaimer
- **Charts**: Candlestick, Line, Histogram, Drawdown (pyqtgraph)
- **Live Ticks**: WebSocket updates in watchlist

12.3 Architecture

QMainWindow → QTabWidget + QSplitter → Chart/Dashboard widgets

Splash screen → Health check API → Fallback to local SQLite

13. Ollama Integration

13.1 Preparation

Install and start Ollama:

```
ollama serve  
ollama pull gemma4:latest
```

13.2 API

```
GET /api/v1/ollama/models  
POST /api/v1/ollama/chat  
POST /api/v1/ollama/generate
```

13.3 Prompt Optimizer

The chat tab includes a built-in prompt optimizer with 5 tips for better trading prompts.

14. Configuration

14.1 Environment Variables

Variable	Default	Description
LML_HOST	127.0.0.1	API server host
LML_PORT	8322	API server port
LML_DB	./data/marketlab.db	SQLite path
OLLAMA_HOST	http://localhost:11434	Ollama server
LML_CORS_ORIGINS	*	CORS origins (comma-separated)

14.2 Demo Data

```
lml demo
```

Loads synthetic prices (IWDA, EIMI, AGGH, BTC) and a demo portfolio.

15. FAQ

Q: Why no real market data? A: Privacy. Real data requires API keys and sends queries to external servers. LML works completely offline.

Q: Can I import my own CSV files? A: Yes, with `lml import txn` and `lml import prices`.

Q: Is this financial advice? A: No. LML is exclusively for research and education. No buy/sell recommendations.

Q: Why Python and not C++/Rust? A: Readability, reproducibility, easy installation. Performance is sufficient for most use cases.

Q: Does this work on macOS/Linux? A: Yes, the web UI and CLI are platform-independent. The Windows app is Windows-specific.

16. Troubleshooting

Problem	Solution
429 Too Many Requests	Rate limiter active — 100 requests/minute/IP
incomplete for FX	Missing exchange rate — import manually or avoid currency
Circular Import	Use <code>state.py</code> , not <code>workspace.py</code> directly
game_id not found	Singleton problem — restart API server
App won't start	Check API server: <code>curl http://127.0.0.1:8322/api/v1/health</code>

Private first. No cloud. No compromises.

Repository: github.com/mrmixx-max/local-market-lab